



**BILKENT UNIVERSITY
EE-102 TERM PROJECT**

**HAND GESTURE-BASED WIRELESS CONTROL
OF ROBOTIC ARM**

**Ahmet Sait İşbilir
Student ID: 22402176
Section: 1**

Objective

The aim of the project is to remotely control the robot arm by processing gyro sensor data using BASYS3. In addition, the angle data will be obtained by IMU sensor and transmitted by RF (HC-05) module through Arduino and each data will be processed and mapped for the robot arm via FPGA Board so that every simple hand gesture will be simulated on the robotic arm.

Design Specifications

In this project, the main goal is to observe 3 different motions at robotic arm by hand gestures. To perform this goal, we have a design which consists of 2 finite state machines for which UART protocol is controlled, and bit transmission as well as bit packaging progressed by them, angle-mapping module, and PWM (servo) driver. Additionally, acquiring angle data and transmitting to FPGA board done by Arduino Nano microprocessor (Atmega-328). Every angle data (pitch angle, yaw angle, roll angle) will be transmitted with 8-bit registers. Each angle value is transferred into 8-bit data which will be transmitted by the RF Bluetooth module (HC-05). Moreover, transferring the angle data, another 8-bit data will be sent to the BASYS3 called “check_sum”. The purpose of this data is to check whether every transmitted data is equivalent to the processed data in FPGA or not in case any unexpected issues may affect the system.

Data transmission will be done at 9600 baud-rate. Each 8-bit angle value will be transferred into 8-bit registers in FPGA by UART State Machine. In UART Protocol, the active bit called “idle state” shows the receiver didn’t get any data from the transmitter, nor permission was given by the designer. When the received bit passed from active to low (idle state settled as active), UART protocol will be passed to the start state and counter begins to count (every bit counting will be done in certain period as clock frequency divided by baud rate). When the counter reaches almost half of the period, the machine passes to the next state called “data state”. In this state, received data will be converted into register, and each bit will be passed to the most significant bit of the register while

counting the bits from 0 to 7. When the register transferring finished and counter reaches almost end of the period, the state machine comes to the “stop state” and resets every configuration for idle state. The state diagram of the UART State machine is given in Figure 1.

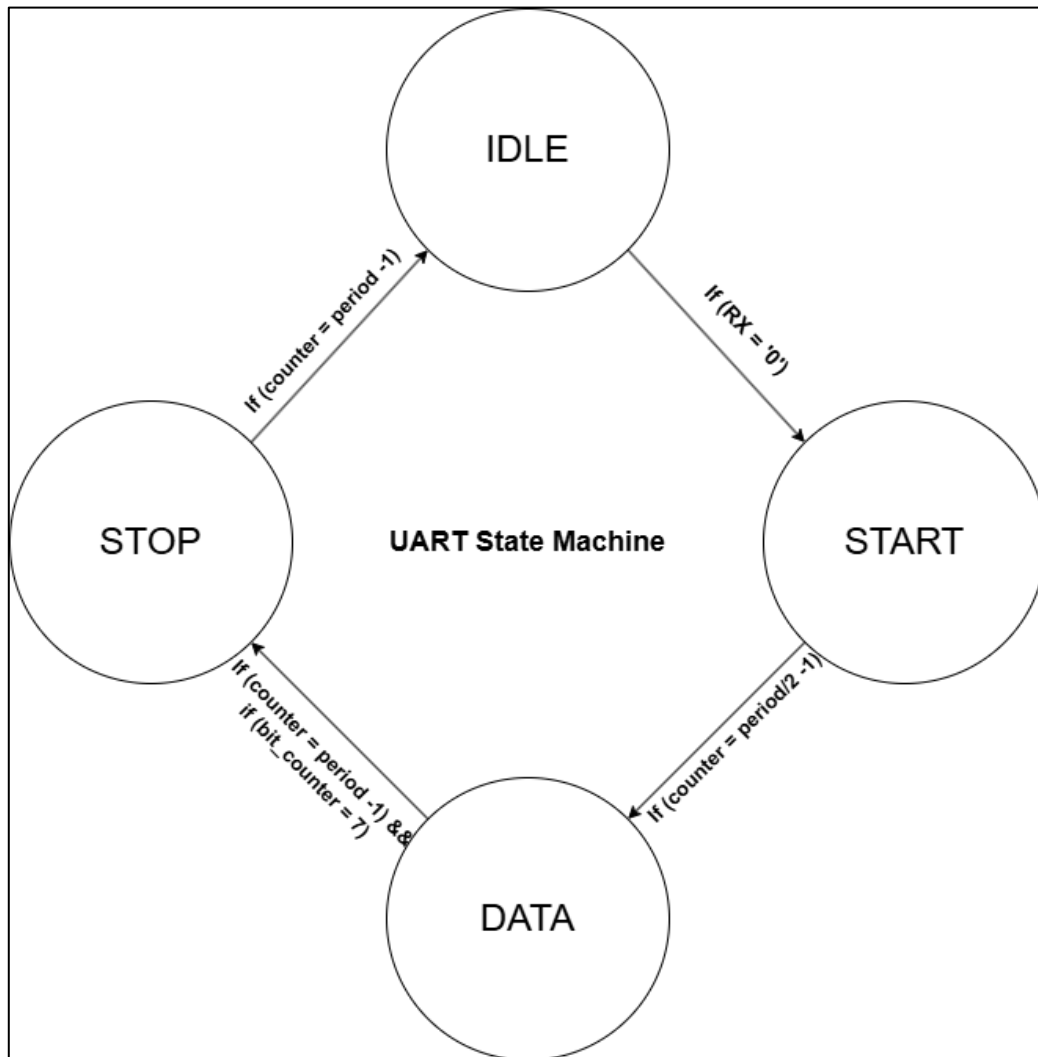


Figure 1: UART State Machine Diagram

Furthermore, to be able to differentiate the angle value, another state machine called “packet_fsm” will be needed. In this machine, each processed 8-bit register will be obtained and stored in corresponding angle variables. Since every angle value and check-sum value will be sent in a certain order, the same order applied in packaging in VHDL is shown in the state diagram (Figure 2). To check whether

the receiver works or not, the starting value (xA5) will be sent via Arduino. When the machine receives that value, it starts to pass each angle registers to the corresponding variables.

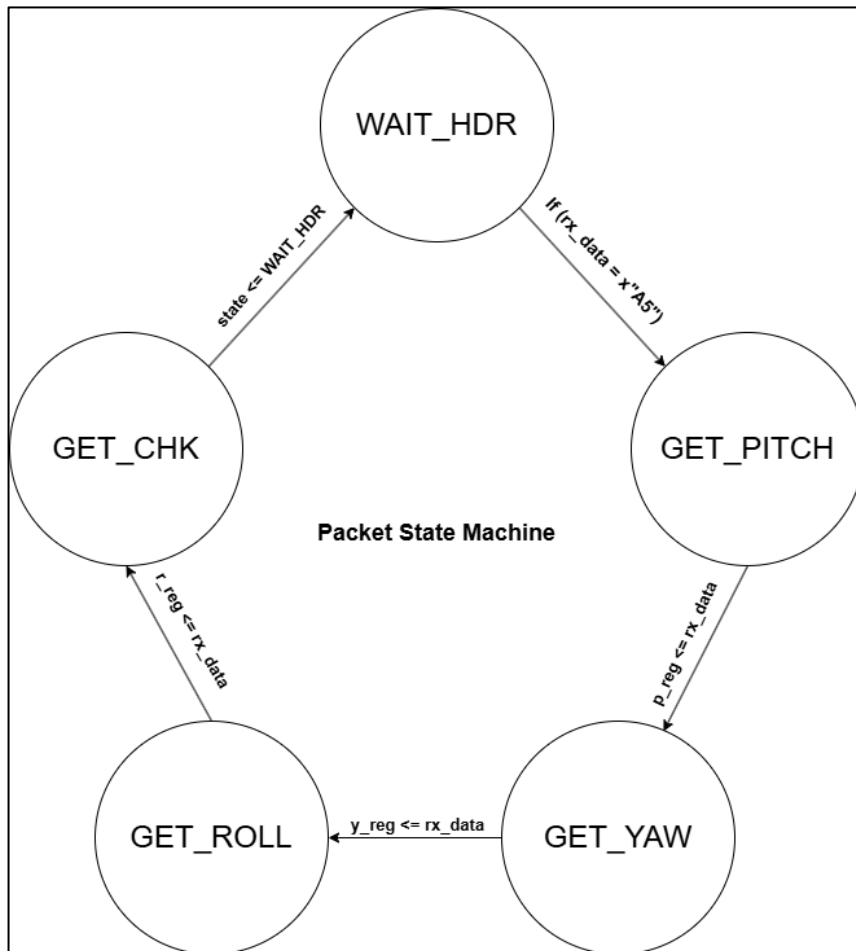


Figure 2: Packet_fsm State Diagram

After each corresponding value is passed to the variables, the check-sum data will be compared in FPGA in case any noise or disturbance interfere with our system. If this comparison satisfies, each angle data will be sent to the angle mapping and PWM-servo modules to control the arm.

In angle mapping module, every raw register converted to unsigned integer value (from 0 to 180) with setting boundaries in case unexpected values might occur. Then our mapped value passed to the PWM module and converted into the servo motor's pulse width. Servo's maximum duty cycle limit is set to 20 milliseconds. Each RTL Scheme is shown in Figure 3.1, 3.2, 3.3, 3.4, and 3.5.

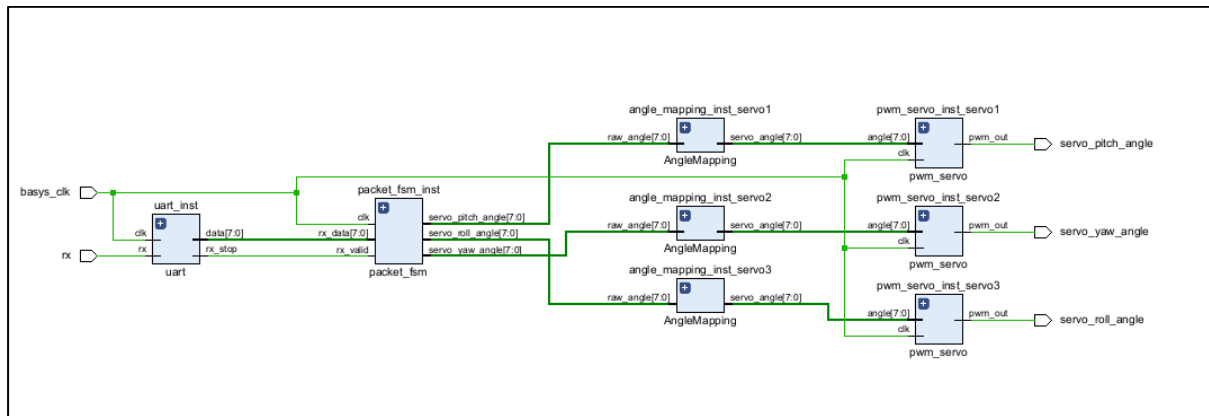


Figure 3.1: RTL Schematic of the System

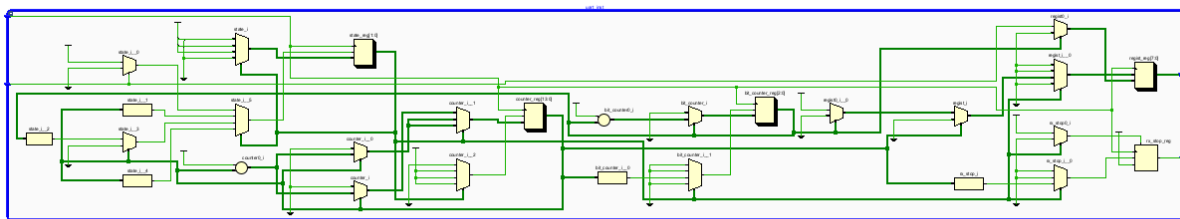


Figure 3.2: RTL Schematic of UART Module

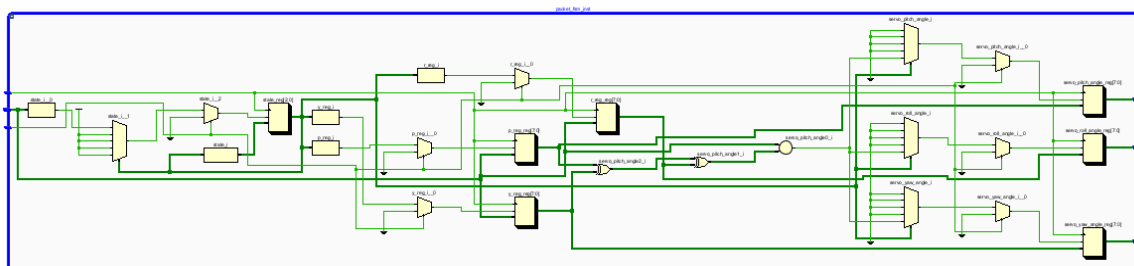


Figure 3.3: RTL Schematic of Packet_fsm Module

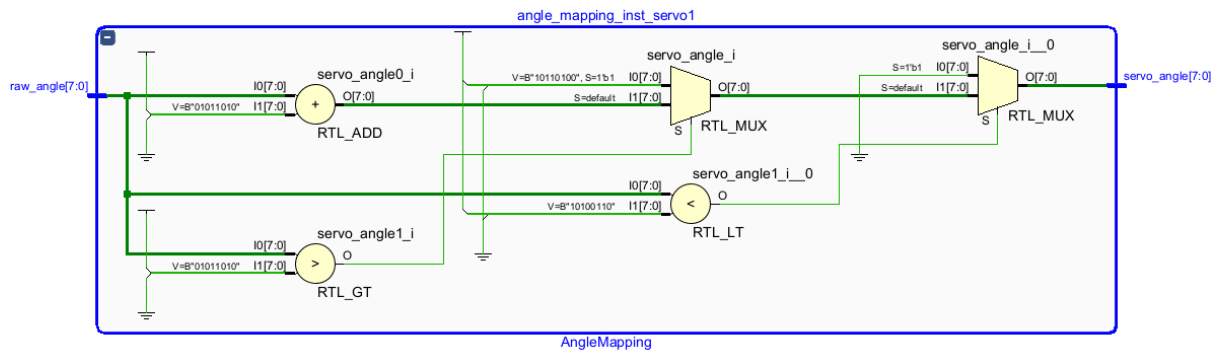


Figure 3.4: RTL Schematic of angle_mapping module

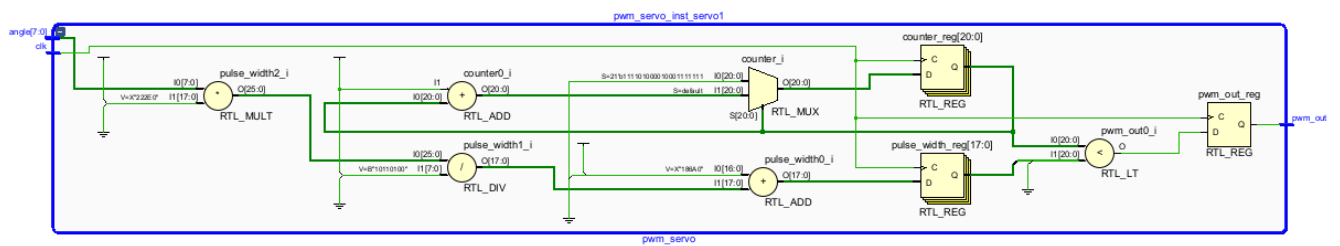


Figure 3.5: RTL Schematic of pwm_servo module

Methodology

First, to understand the UART Protocol and some properties of it, some investigations made through the period and initial finite state machine designs performed. After that, Arduino breadboard connections and a few pull-down resistors were done. To perform the communication, 2 RF modules (HC-05) calibrations were done by powering the enable pin and activating AT-Mode. AT-Mode is a calibration state which enables us to change some of the properties of RF modules. Therefore, the receiver and transmitter (master-slave) configuration were set and programmed. Then, whole VHDL modules were written and tested

with 1 servo motor. Once the test was successful, mechanical parts were integrated into servo motors. While some servo motors cannot handle exact weight, power performance was increased. After some testing, the Arduino system was attached to the glove. Finally, the whole system was demonstrated.

Results

As a result, our project was performed successfully. Every hand gesture passed to the arm, and the arm reflected every hand gesture perfectly. BASYS3 Pinout as well as each hand gesture and the reflection of the arm are shown in certain figures (4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, and 4.9)

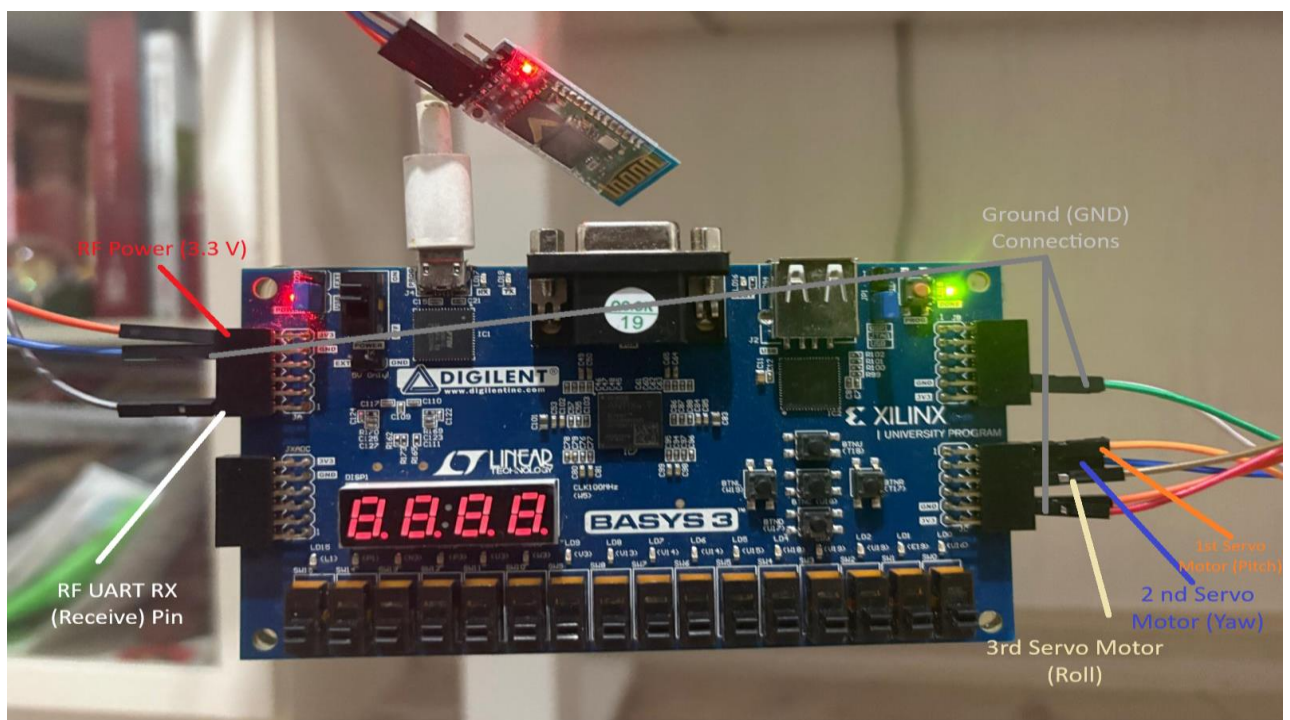


Figure 4.0: BASYS3 Connections (RF Module, Servo motors, power connections)

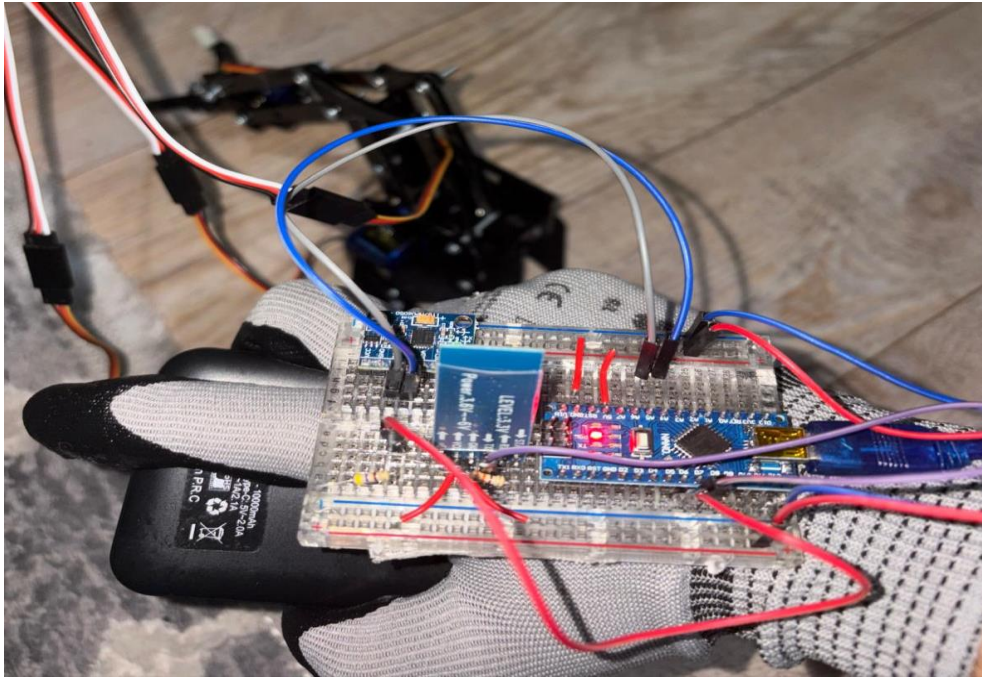


Figure 4.1: Robotic arm turns to the left as hand gesture points left

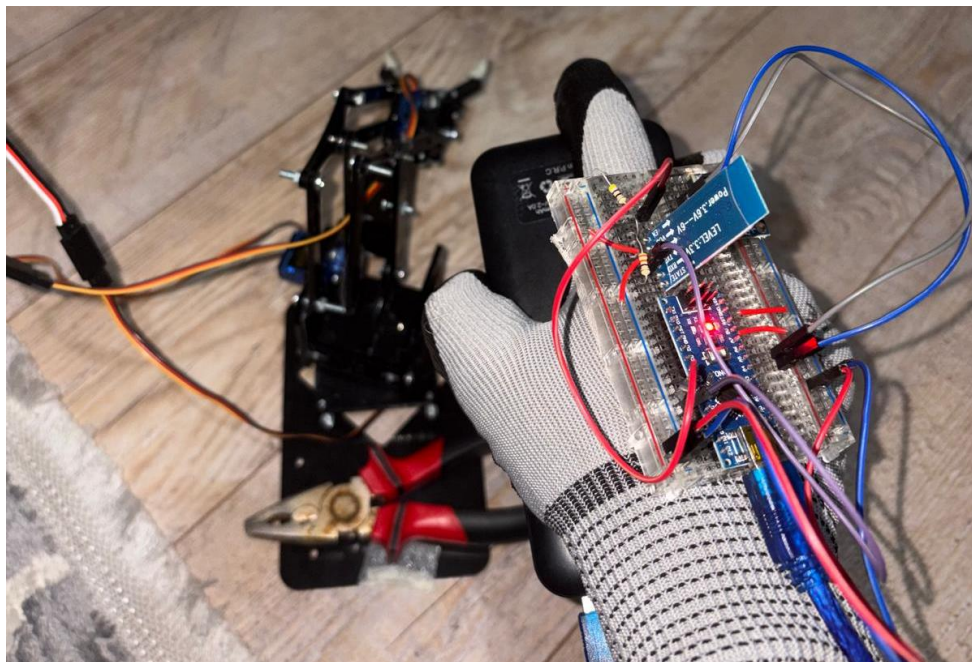


Figure 4.2: Robotic arm turns to the front as hand gesture points front



Figure 4.3: Robotic arm turns to the right as hand gesture points right

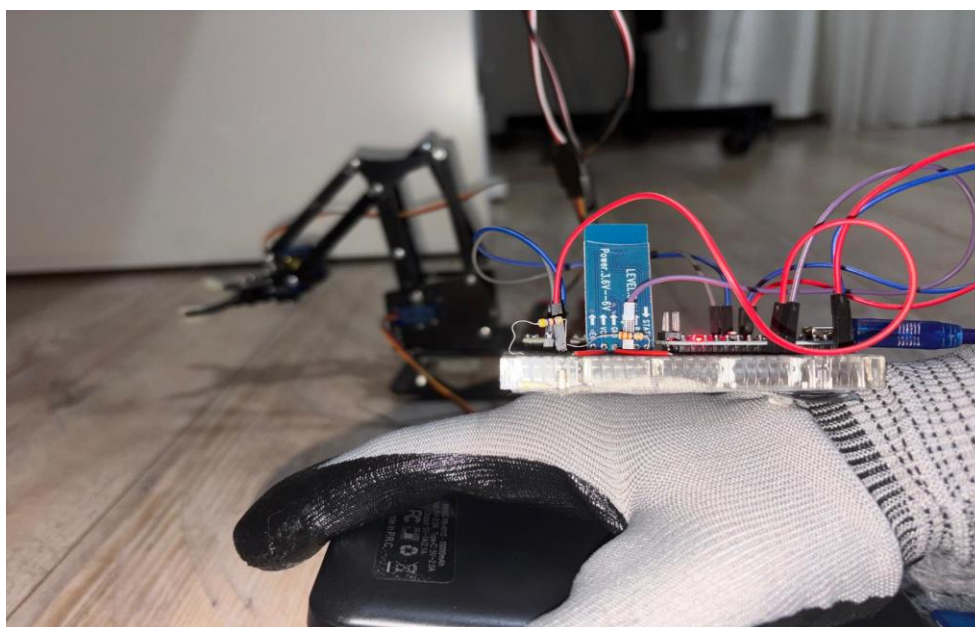


Figure 4.4: Robotic arm in normal state as hand gesture stays in normal state

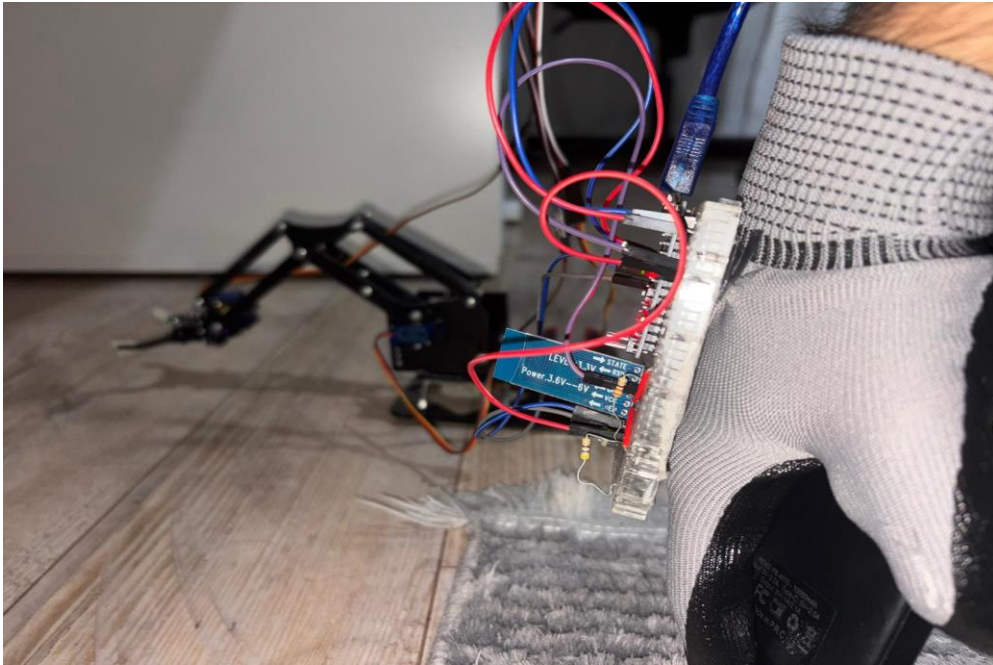


Figure 4.5: Robotic arm goes forward as hand gesture points down.

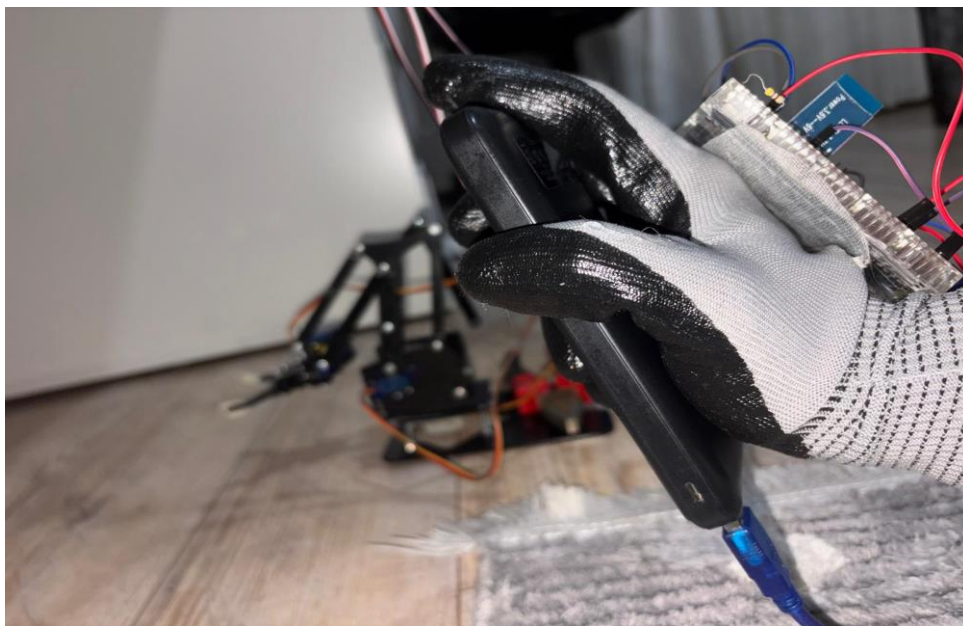


Figure 4.6: Robotic arm goes backward as hand gesture points up.

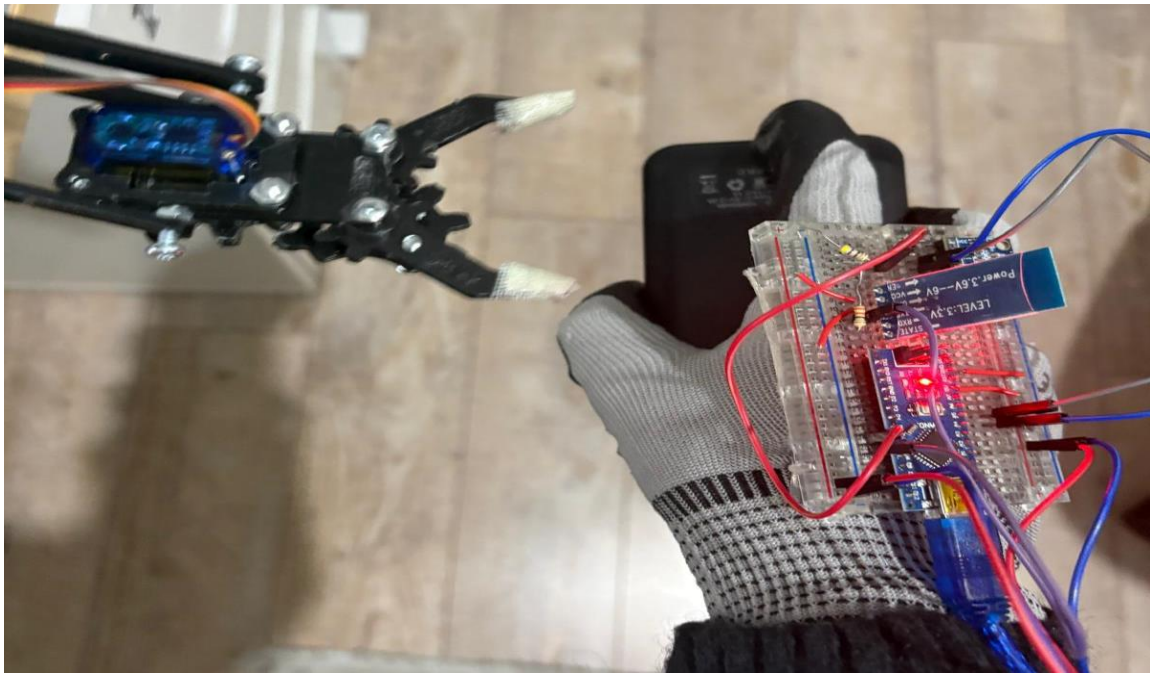


Figure 4.7: Robotic gripper stays open as hand gesture stays normal.



Figure 4.8: Robotic gripper nearly closed as hand gesture rotates to the right.

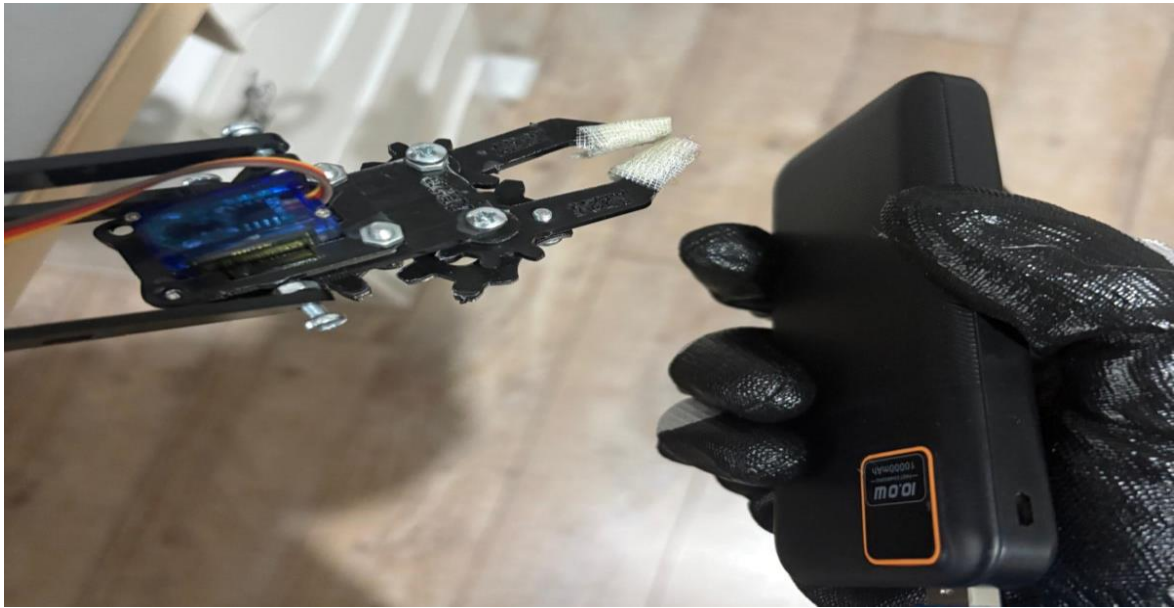


Figure 4.9: Robotic gripper completely closed as hand gesture rotates fully to the right.

Conclusion

In summary, the main objective of the project was remotely controlling the robot arm by processing gyro sensor data using BASYS3. It was verified that the expectations from this project were satisfied by the triumphant result. Every motion obtained, processed for mechanical motions. As a result, certain motions of hand were reflected in robotic arm, and every finite state machine for communication protocols performed their algorithms. Through this experience, I achieved numerous abilities in signal processing and communication through FPGA systems as well as designing finite state machines. Even though designing UART machines was difficult work which I didn't encounter before, I managed to improve myself and implement my imagination into the real world.

References

Mehmet Burak Aykenar. (2021). *VHDL ile FPGA PROGRAMLAMA - Ders18: UART Receiver Tasarımı ve Simülasyonu* [Video]. YouTube.

<https://youtu.be/d7XPkZDWWQY>

Russell. (2022, June 14). *What is a UART?* Nandland.

<https://nandland.com/what-is-a-uart/>

Appendices

GyroTransmitter.cpp

```
#include <MPU6050_tockn.h>
```

```
#include <Wire.h>
```

```
#include <SoftwareSerial.h>
```

```
MPU6050 mpu6050(Wire);
```

```
SoftwareSerial hc05(7, 6);
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    hc05.begin(9600);
```

```
    Wire.begin();
```

```
    mpu6050.begin();
```

```
    mpu6050.calcGyroOffsets(true);
```

```
}
```

```

void loop() {
    static float yaw_angle = 0;
    static unsigned long lasttime = 0;

    unsigned long now = millis();
    float dt = (now-lasttime) /1000.0;
    lasttime = now;
    mpu6050.update();

    float angle_x = mpu6050.getAngleX() *0.7;
    float angle_y = mpu6050.getAngleY() *0.4;
    float z_rate = mpu6050.getGyroZ()*0.7;

    yaw_angle += z_rate * dt;

    yaw_angle = constrain(yaw_angle, -90, 90);

    int8_t ax = (int8_t)constrain(angle_x, -90, 90);
    int8_t ay = (int8_t)constrain(angle_y, -90, 90);
    int8_t az = (int8_t)yaw_angle;

    int8_t chk = ax^ay^az;

    hc05.write(0xA5);
    hc05.write(ax);
    hc05.write(ay);
    hc05.write(az);
    hc05.write(chk);

```

```
delay(20);
```

```
}
```

top.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity top is
```

```
Port (
```

```
    basys_clk: in std_logic;
```

```
    rx: in std_logic;
```

```
    servo_pitch_angle: out std_logic;
```

```
    servo_roll_angle: out std_logic;
```

```
    servo_yaw_angle: out std_logic
```

```
);
```

```
end top;
```

```
architecture Behavioral of top is
```

```
    signal raw_angle: std_logic_vector(7 downto 0);
```

```
    signal s1, s2, s3: std_logic_vector(7 downto 0);
```

```
    signal angle1, angle2, angle3: unsigned(7 downto 0);
```

```
    signal rx_valid: std_logic;
```

```
begin
```

```
uart_inst: entity work.uart
```

```
  port map(
    rx => rx,
    clk => basys_clk,
    data => raw_angle,
    rx_stop => rx_valid
  );
```

```
packet_fsm_inst: entity work.packet_fsm
```

```
  port map(
    clk => basys_clk,
    rx_data => raw_angle,
    servo_pitch_angle => s1,
    servo_yaw_angle => s2,
    servo_roll_angle => s3,
    rx_valid => rx_valid
  );
```

```
angle_mapping_inst_servo1: entity work.AngleMapping
```

```
  port map(
    raw_angle => signed(s1),
    servo_angle => angle1
  );
```

```
pwm_servo_inst_servo1: entity work.pwm_servo
```

```
  port map(
    clk => basys_clk,
    angle => angle1,
    pwm_out => servo_pitch_angle
  );
```

```
angle_mapping_inst_servo2: entity work.AngleMapping
  port map(
    raw_angle => signed(s2),
    servo_angle => angle2
  );
```

```
pwm_servo_inst_servo2: entity work.pwm_servo
  port map(
    clk => basys_clk,
    angle => angle2,
    pwm_out => servo_yaw_angle
  );
```

```
angle_mapping_inst_servo3: entity work.AngleMapping
  port map(
    raw_angle => signed(s3),
    servo_angle => angle3
  );
```

```
pwm_servo_inst_servo3: entity work.pwm_servo
  port map(
    clk => basys_clk,
    angle => angle3,
    pwm_out => servo_roll_angle
  );
```

```
end Behavioral;
```

uart.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity uart is
```

```
    generic (
```

```
        clk_freq    : integer := 100_000_000;
```

```
        baud_rate   : integer := 9600
```

```
    );
```

```
    Port (
```

```
        clk : in std_logic;
```

```
        rx  : in std_logic;
```

```
        data: out std_logic_vector(7 downto 0);
```

```
        rx_stop: out std_logic
```

```
    );
```

```
end uart;
```

```
architecture Behavioral of uart is
```

```
    type states is (S_IDLE, S_START, S_DATA, S_STOP);
```

```

signal state : states := S_IDLE;

constant period : integer:= clk_freq/baud_rate;

signal counter: integer range 0 to period;
signal bit_counter: integer range 0 to 7;
signal regist: std_logic_vector (7 downto 0);

begin
  process(clk) begin
    if (rising_edge(clk)) then

      case state is

        when S_IDLE =>
          rx_stop <= '0';
          if (rx = '0') then
            state <= S_START;
          end if;

        when S_START =>
          if counter = period/2 -1 then
            state <= S_DATA;
            counter <= 0;
          else

```

```

        counter <= counter + 1;
    end if;
when S_DATA =>

    if (counter = period -1) then
        if (bit_counter = 7) then
            state <= S_STOP;
            bit_counter <= 0;
        else
            bit_counter <= bit_counter + 1;
        end if;

        regist(bit_counter) <= rx;
        counter <= 0;
    else
        counter <= counter + 1;
    end if;
when S_STOP =>
    if (counter = period -1) then
        state <= S_IDLE;
        counter <= 0;
        rx_stop <= '1';
    else
        counter <= counter + 1;
    end if;
end case;

```

```

        end if;
    end process;

    data <= regist;
end Behavioral;

```

packet_fsm.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity packet_fsm is
    Port (
        clk      : in std_logic;
        rx_valid  : in std_logic;
        rx_data   : in std_logic_vector(7 downto 0);

        servo_pitch_angle : out std_logic_vector(7 downto 0);
        servo_yaw_angle   : out std_logic_vector(7 downto 0);
        servo_roll_angle  : out std_logic_vector(7 downto 0)
    );
end packet_fsm;

architecture Behavioral of packet_fsm is

```

```

type state_type is (WAIT_HDR, GET_PITCH, GET_YAW, GET_ROLL,
GET_CHK);

```

```

signal state : state_type := WAIT_HDR;

```

```

signal p_reg, y_reg, r_reg : std_logic_vector(7 downto 0);

```

```

begin

```

```

process(clk)

```

```

begin

```

```

if rising_edge(clk) then

```

```

if rx_valid = '1' then

```

```

case state is

```

```

when WAIT_HDR =>

```

```

if rx_data = x"A5" then

```

```

state <= GET_PITCH;

```

```

end if;

```

```

when GET_PITCH =>

```

```

p_reg <= rx_data;

```

```

state <= GET_YAW;

```

```

when GET_YAW =>

```

```

y_reg <= rx_data;

```

```

state <= GET_ROLL;

```

```

when GET_ROLL =>
    r_reg <= rx_data;
    state <= GET_CHK;

when GET_CHK =>
    if rx_data = (p_reg xor y_reg xor r_reg) then --check
        servo_pitch_angle <= p_reg;
        servo_yaw_angle  <= y_reg;
        servo_roll_angle <= r_reg;
    end if;
    state <= WAIT_HDR;

end case;

end if;

end if;

end process;

end Behavioral;

```

AngleMapping.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

entity AngleMapping is

Port (

raw_angle: in signed(7 downto 0); -- (-128,128) => -90, 90

servo_angle: out unsigned(7 downto 0) -- (0, 256) => 0, 180

);

end AngleMapping;

architecture Behavioral of AngleMapping is

begin

process(raw_angle)

variable a : integer range -128 to 127;

begin

a := to_integer(raw_angle);

if a < -90 then

servo_angle <= to_unsigned(0, 8);

elsif a > 90 then

servo_angle <= to_unsigned(180, 8);

else

servo_angle <= to_unsigned(a + 90, 8);

end if;

end process;

end Behavioral;

pwm_servo.vhd

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity AngleMapping is

Port (

raw_angle: in signed(7 downto 0); -- (-128,128) => -90, 90

servo_angle: out unsigned(7 downto 0) -- (0, 256) => 0, 180

);

end AngleMapping;

architecture Behavioral of AngleMapping is

begin

process(raw_angle)

variable a : integer range -128 to 127;

begin

a := to_integer(raw_angle);

if a < -90 then

```

        servo_angle <= to_unsigned(0, 8);
    elsif a > 90 then
        servo_angle <= to_unsigned(180, 8);
    else
        servo_angle <= to_unsigned(a + 90, 8);
    end if;
end process;

end Behavioral;

```

cons.xdc

```

set_property PACKAGE_PIN W5 [get_ports basys_clk]
set_property IOSTANDARD LVCMOS33 [get_ports basys_clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
basys_clk]

```

```

set_property PACKAGE_PIN J1 [get_ports {rx}]
set_property IOSTANDARD LVCMOS33 [get_ports {rx}]

```

```

set_property PACKAGE_PIN K17 [get_ports {servo_pitch_angle}]
set_property IOSTANDARD LVCMOS33 [get_ports {servo_pitch_angle}]

```

```
set_property PACKAGE_PIN M18 [get_ports {servo_roll_angle}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {servo_roll_angle}]
```

```
set_property PACKAGE_PIN N17 [get_ports {servo_yaw_angle}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {servo_yaw_angle}]
```